

Here is your build plan, grounded in the real brief and a live audit of your actual Customer-Churn.csv. Key facts I verified just now:

- 7,043 rows × 21 columns, zero duplicate rows, all 7,043 customerIDs unique.
- Class balance: 73.5% No / 26.5% Yes churn (≈2.8:1 — moderately imbalanced, so accuracy is a trap).
- TotalCharges is stored as a string, not a number — coercing it creates 11 nulls, and every one of those rows has tenure = 0 (brand-new customers, no bill yet). That's the dataset's famous hidden defect.
- No classic leakage column exists (no cancellation_date etc.), so leakage hunting here is about the label proxy logic + the "No internet service" vs "No" encoding trap.

The 20 Sequential Tasks

Phase A — Foundation (Tasks 1–2)

Task 1: Repo scaffold, virtualenv, git init

- Goal: project/ layout, .venv, requirements.txt, .gitignore, git init, first commit.
- Concepts: Python environment isolation — why a venv: your project must pin versions that actually reproduce (requirements.txt with pandas==2.x, scikit-learn, streamlit, xgboost/lightgbm, groq, shap). Incremental git — the brief grades "meaningful, incremental commits," so commit 1 is the scaffold itself.
- Decisions: This is a pure-data science + agent repo, not a web service, so no src/ package complexity — flat modules (data.py, model.py, tools.py, agent.py, app.py) per the guide's layout. Python 3.14 is what's installed here; pin libraries that support it.
- Architecture: Creates the four-layer boundary skeleton before any code exists, so layers never bleed into each other.
- Theory: Separation of concerns / modularity — the brief's "not one giant file" is really testing whether you understand dependency direction (UI → agent → tools → model/data, one-way).

Task 2: Raw data audit against the brief

- Goal: Load the CSV, assert what you know (shape, dtypes, class balance, duplicates), and write down the three problems you found — before cleaning anything.
- Concepts: Data profiling — dtype inspection catches the TotalCharges string bug instantly; .isnull().sum() shows nothing because the 11 blanks are empty strings, not NaN

— this is why you must coerce then re-check. Class imbalance measured first (26.5%) because it dictates every later metric decision.

- Decisions: Record findings in a DATA_NOTES.md or notebook section now — this becomes the "documented cleaning decisions" the brief requires, and it's what you'll cite in the follow-up interview.

- Architecture: Pure read-only pass; nothing is mutated yet. This is the input contract the data layer must satisfy.

- Theory: Garbage in, garbage out + the difference between structural defects (wrong dtype) and semantic defects (blank = "no billing yet"). The grading checks whether you can name a concrete defect — you now have one: TotalCharges blank on tenure=0 rows.

Phase B — Data Layer (Tasks 3–6)

Task 3: EDA — distributions, cardinality, encodings

- Goal: Histograms/boxplots of tenure, MonthlyCharges, TotalCharges; value counts of every categorical; correlation of each feature with Churn.

- Concepts: Univariate vs bivariate analysis, cardinality (too many unique values = useless categorical feature), encoding trap: columns like MultipleLines contain "No" and "No phone service" — two distinct string values that semantically both mean "no." Same for the six internet add-ons.

- Decisions: Decide the canonical encoding: collapse "No internet service" → "No" for internet add-ons; treat "No phone service" in MultipleLines as its own value or merge into No. Document this — it's a cleaning decision graders look for.

- Architecture: Outputs feed the feature-engineering step; also gives the agent's dataset tool real "distribution" answers to draw from later.

- Theory: Feature encoding theory — you're deciding between One-Hot, Label, or Ordinal encoding based on ordinality: Contract is ordinal (Month-to-month < One year < Two year), PaymentMethod is nominal.

Task 4: Missingness + leakage deep-dive

- Goal: Characterize the 11 TotalCharges nulls, and hunt explicitly for label leakage.

- Concepts: Missingness mechanisms (MCAR/MAR/MNAR) — here the blanks are not random: missing billing correlates perfectly with tenure=0, i.e., new customers. That's a finding, not a bug. Label leakage — a feature that contains future/post-label information;

none of the classic ones (`cancellation_date`, `days_since_last_login`, `account_status`) exist, but you must write that reasoning down so the grader sees you looked.

- Decisions: Two defensible options for the 11 rows: (a) fill `TotalCharges` = `MonthlyCharges` × tenure (= `MonthlyCharges` when tenure=0, or 0), or (b) drop them ($11/7043 \approx 0.16\%$). Pick one, justify, document.

- Architecture: These decisions live in `data.py`'s `clean_data()`, with a returned log of (issue, action, reason) — the audit trail the whole project points to.

- Theory: Leakage theory — why it inflates metrics: the model memorizes the label via a proxy and fails on real deployment data. Your write-up of "I checked for X, found none, and here's why `TotalCharges` is safe" is exactly what the interview probes.

Task 5: Build the cleaning pipeline in `data.py`

- Goal: Productionize Tasks 3–4 into `load_data()` and `clean_data()` — deterministic, tested, returns (`clean_df`, `cleaning_log`).

- Concepts: Data pipeline engineering — idempotency (running twice gives the same result), reproducibility (seed everything), and pipeline-as-documentation (each step is a named function with a comment that records the decision).

- Decisions: Hardcode the fixes (encoding map, `TotalCharges` coercion rule) rather than auto-inference — explicit > clever for a take-home, and it's what you can defend.

- Architecture: This is the ONLY module that touches raw data. `model.py` and `tools.py` import from it; they never re-clean. The single-responsibility boundary is the proof of "modular code."

- Theory: Extract-Transform-Load (ETL) discipline applied to a `DataFrame`; and functional programming ideas (pure functions, no hidden global state) so the agent's code-execution tool can't corrupt shared state.

Task 6: Train/test split (stratified, leakage-aware)

- Goal: `train_test_split(test_size=0.2, stratify=Churn)` with a fixed seed, saved so predict-time uses the same preprocessing.

- Concepts: Overfitting vs generalization — the test set is a stand-in for the future; stratification preserves the 73.5/26.5 ratio in both splits so the minority class isn't accidentally absent from one; random seed makes results reproducible and defensible.

- Decisions: Where to split in the pipeline order: always after cleaning/feature engineering, before any model training, and never tune hyperparameters on the test set (that would be

indirect leakage). No train/test leakage here because you do no dataset-level scaling/fitting in cleaning.

- Architecture: The split is a data.py responsibility; model.py consumes the prepared (X_train, y_train, X_test, y_test) — keeping train/test concerns in the data layer keeps the agent tool from ever seeing test rows.

- Theory: Bias-variance tradeoff and why a held-out set is the honest estimator of performance; stratify guards against the rare-class problem where a 26.5% class could randomly vanish from a small test fold.

Phase C — Model Layer (Tasks 7–11)

Task 7: Metric selection — the written justification

- Goal: Write, in prose, why accuracy is wrong and which metric you'll report — before training anything.

- Concepts: Class-imbalance math: a model that always predicts "No" scores 73.5% accuracy while catching 0% of churners. Precision/Recall/F1: precision = of those I flagged, how many really churned; recall = of those who churned, how many I caught. PR-AUC vs ROC-AUC: PR-AUC is sensitive to the minority class, ROC-AUC can look great on imbalanced data while being useless. Business cost asymmetry: a missed churner (false negative) costs a customer; a false positive only wastes a retention offer — so lean recall.

- Decisions: Primary metric = F1 on the churn class (and PR-AUC as a secondary), with precision and recall reported separately so the tradeoff is visible, not hidden in one number. Write this paragraph into the README now.

- Architecture: The metric choice is enforced at evaluation time in model.py, not scattered in the notebook.

- Theory: Cost-sensitive learning — metrics are a proxy for business utility; choosing a metric is choosing an assumption about what a mistake costs. That reasoning is what the interview asks about.

Task 8: Trivial + linear baselines

- Goal: Train (a) majority-class predictor, (b) LogisticRegression, report both.

- Concepts: Baselines are the null hypothesis — you can't claim your model is good without a reference. Majority-class gives the accuracy floor (73.5%) and a recall-of-0 reference; logistic regression gives a simple interpretable benchmark.

- Decisions: Use `class_weight='balanced'` in `LogisticRegression` so the imbalance is addressed at training time, not just in the metric.
- Architecture: A `train_baseline()` function in `model.py`; results logged to a metrics dict for the notebook and README table.
- Theory: Bias-variance + linearity assumption: logistic regression assumes a linear decision boundary in log-odds space — if trees beat it later, you've empirically demonstrated the data is non-linear, which is a defensible claim in the write-up.

Task 9: Tree ensemble with class weighting

- Goal: `RandomForest` or `XGBoost/LightGBM` with `scale_pos_weight/class_weight='balanced'`; modest hyperparameter tuning (a grid or randomized search on a handful of params only).
- Concepts: Ensembles — bagging (`RandomForest`) reduces variance by averaging many trees; boosting (GBM) reduces bias sequentially. Class weighting reweights the loss so the minority class errors cost more. Hyperparameter tuning — doing a small, seeded search (e.g., 20–50 random draws on `n_estimators`, `max_depth`, `learning_rate`) is enough; you're not chasing a leaderboard.
- Decisions: Which library? Start with `LightGBM` or `XGBoost` if install is painless; else `RandomForest` is fully sufficient per the guide. Tune on the training/validation split, never the test set.
- Architecture: `train_model()` returns (model, metrics); the tuned model is persisted (joblib) so the agent and app load it without retraining.
- Theory: Gradient boosting mechanics — each tree fits the residual of the previous ensemble, which is why boosting squeezes out bias on exactly the hard-to-classify (often churning) rows; that's the theoretical reason GBM tends to beat RF on tabular data.

Task 10: Evaluate + explainability (SHAP or `feature_importances_`)

- Goal: Full evaluation on test set; produce both global and per-prediction explanations.
- Concepts: Confusion matrix (TP/FP/FN/TN) — read it as a business table, not a 2×2 grid: how many real churners did I miss? Thresholding — `sklearn` gives probabilities; you choose the cutoff that balances recall/precision (e.g., tune threshold on validation). SHAP — per-prediction attribution: "this customer is at 74% because `contract=monthly` (+0.21), `tenure=2` (+0.18)..."

- Decisions: Try SHAP (TreeExplainer) — it's one dependency and a few lines for tree models. If it fights your install, fall back to `feature_importances_` and say so explicitly in the README (per the guide, honesty about this beats pretending).
- Architecture: Explainability is a separate function `explain(prediction, row)` -> `top_factors` inside `model.py`; this is what feeds `top_factors` in the model tool.
- Theory: Interpretability vs explainability — global feature importance says "what generally matters"; SHAP says "why THIS prediction." The brief's "why is this customer high-risk" question requires the per-row kind.

Task 11: `predict_churn_risk()` — the model as a callable tool

- Goal: The single public function: `predict_churn_risk(customer_id: str) -> {risk_score, top_factors}` — and an overload accepting a raw feature dict (for hypotheticals).
- Concepts: Function-as-API contract — the agent (and app) must never touch the raw model; they call this. ID lookup vs raw input: given an ID, load the row from the cleaned df, preprocess, predict; given a dict, validate the keys match the feature schema, preprocess identically. Same preprocessing path both ways — this is how you guarantee a hypothetical point is "evaluated against the original data" (brief's words).
- Decisions: Output shape is exactly `{"risk_score": float in [0,1], "top_factors": [{"feature", "contribution"}], "customer_id", "prediction_class"}` — JSON-serializable, deterministic, and validated for sanity (`risk_score` clamped/asserted in 0,1) so the agent's verify step has something to check.
- Architecture: This is the model-layer → tool-layer seam. `tools.py` will wrap this function with a description; nothing else about the model is exposed.
- Theory: Interface segregation — the narrowest possible public surface; dependency inversion — the agent depends on an abstraction (a callable), not the concrete model internals.

Phase D — Tool Layer (Tasks 12–13)

Task 12: The restricted code-execution tool

- Goal: Let the agent answer arbitrary aggregation/correlation/filter questions by executing short pandas snippets against df in a locked namespace.
- Concepts: Sandboxing — you `exec()` code in a namespace containing ONLY df and a whitelist of safe builtins (`len`, `sum`, `min`, `max`, `abs`, `round`, `range`, `sorted`, `isinstance`, `float`, `int`, `str`, `list`, `dict`, `tuple`, `set`, `bool`, `type`, `enumerate`, `zip`). Block `__builtins__` wholesale, forbid

import, open, os, subprocess, eval, exec, __globals__, attribute traversal beyond safe methods. Why not just trust the LLM? Because a mis-generated snippet is the single most likely source of a crash or a fabricated number.

- Decisions: Capture stdout/stderr AND the returned value; return a truncated summary (first N rows, df.describe()-style) for large results, plus any error string — the agent needs both the number and the error to self-check. Document in the README that this is local-take-home-appropriate, not production-hardened — naming the tradeoff is scored.

- Architecture: Lives in tools.py, isolated from the agent loop; returns {"status": "ok"|"error", "output": ..., "error": ...}.

- Theory: Code-as-tool vs fixed-function design: exec is strictly more general (any EDA question works) at the cost of safety; this is the classic expressiveness vs safety tradeoff, and the brief explicitly asks for the exec approach.

Task 13: Tool registry + output contract

- Goal: A single TOOLS registry: each tool is {name, description, parameters_schema, function, output_normalizer} — the only interface the agent sees.

- Concepts: Tool-calling schema (OpenAI/Groq function-calling JSON schema) — the LLM needs precise descriptions to choose tools correctly, so descriptions read like documentation ("predict_churn_risk: compute churn risk for an existing customerID or a dict of features"). Output normalization — every tool returns JSON-serializable output, so the agent's verify step can log and diff them cleanly.

- Decisions: Two tools minimum — predict_churn_risk and run_data_query (the restricted exec). Optionally a third, list_columns/get_schema, to reduce guesswork. Keep the schema strict: additionalProperties: false, required fields — this prevents malformed calls that waste rate-limited tokens on retries.

- Architecture: The registry is the contract layer: agent.py imports TOOLS and never hardcodes tool names; swapping in a new tool requires zero agent changes.

- Theory: Protocol design — the quality of your tool descriptions is what determines whether a small open model can plan correctly; this is prompt engineering as system design, not prompt tweaking.

Phase E — Agent Layer (Tasks 14–16)

Task 14: LLM client module (Groq, thin + swappable)

- Goal: agent.py (or llm.py) wraps the Groq client: one call(messages, tools) -> response function; a parse_tool_call() that either reads native tool_calls or parses JSON fallback.
- Concepts: Native tool-calling vs JSON-prompting fallback — the brief explicitly blesses prompting for structured JSON if your model fights native tool-calling; design the client to attempt native first, fall back to JSON, and validate with Pydantic. Rate-limit efficiency — free tiers throttle tokens/min, so the loop must make the minimum LLM calls per question: decide the plan in one call, then execute tool calls without re-planning from scratch unless the plan actually failed.
- Decisions: Groq Llama 3.3 as default (fast, native tool-calling), provider isolated behind one module so swapping to OpenRouter is a one-line change if you hit limits mid-build.
- Architecture: This module is the ONLY place an API key/endpoint appears; the agent loop calls it generically.
- Theory: Model-as-executor vs model-as-planner — you are using the LLM for reasoning/orchestration, not for arithmetic; every number must come from tools. That separation is the whole anti-hallucination design.

Task 15: Plan → act → observe loop (single-step first)

- Goal: End-to-end for one question: LLM chooses a tool → your code executes it → raw output is returned → LLM writes final answer citing that output.
- Concepts: ReAct-style loop — the model alternates reasoning and tool calls; you keep messages history so the model sees its own prior tool calls and their outputs. Observation capture — store every tool result in a run-level execution_log (tool, args, output, timestamp, status).
- Decisions: Implement as a raw hand-written loop (per the guide's recommendation) — max iterations cap (e.g., 6) to prevent infinite loops and protect the rate limit; on iteration exhaustion, say "I couldn't fully answer" rather than improvising.
- Architecture: agent.run(question) -> {answer, execution_log} — the loop owns state; the UI never sees internals except the log (for the trace panel).
- Theory: Tool-use as grounding — the observation channel replaces parametric memory ("the model knows") with episodic, verifiable evidence ("the tool computed"); this is the theoretical core of why agents hallucinate less on data questions when properly wired.

Task 16: Self-check / anti-hallucination + multi-step planning

- Goal: The verify step — the single most graded requirement — plus a true multi-step question working end-to-end.
- Concepts: Numeric-claim validation: before an answer is returned, a validator scans the final text for numbers (regex), collects tool outputs from `execution_log`, and flags any figure that doesn't appear in some logged output. Failed/empty/nonsensical results (risk score outside 0,1, empty df) trigger a retry or an explicit "couldn't compute" message — never a guess. Multi-step planning: for "which customers are likely to churn, and does that correlate with region?" the loop must: (1) call `predict_churn_risk` over customers, (2) call `run_data_query` to group by region + correlate, (3) verify both outputs, (4) compose the answer from only those two results.
- Decisions: Make the validator a real function (`validate_answer(answer, execution_log) -> {ok, violations}`) — so you can literally point to the line when asked "how do you know it isn't hallucinating?" Retry policy: one retry per failed tool call, then fail loudly.
- Architecture: The verify step sits between the LLM's final draft and the UI; it's a gate, not a suggestion. The critic-agent (stretch) is a second gate you can add later — this validator is the mandatory first one.
- Theory: Grounding and entailment — you're enforcing that every stated fact is entailed by tool evidence; graceful degradation — a system that says "I couldn't compute that" honestly is more trustworthy than one that fabricates. That honesty is the requirement.

Phase F — Interface Layer (Tasks 17–18)

Task 17: Streamlit chat app — live wiring + error boundaries

- Goal: `app.py`: render chat from `session_state`, call `agent.run()`, show responses; wrap everything in `try/except` with a friendly fallback (never a raw traceback).
- Concepts: Session state — Streamlit reruns the script on every interaction; `st.session_state.messages` is how you persist conversation. Error boundaries — a `try/except` around each agent call that maps exceptions to "Sorry, something went wrong computing that — the tool reported: ..." Live wiring — the app must actually call `predict_churn_risk` and the agent, never return canned answers.
- Decisions: Keep `app.py` thin — no model/tool/agent logic in it; it's a presenter. Format model outputs (`risk_score` as a progress bar, `top_factors` as a table) rather than dumping raw dicts.
- Architecture: Strict one-way dependency: `app → agent → tools → model → data`. This is the modularity proof the grader looks for.

- Theory: MVC-ish separation — the chat UI is the view, the agent is the controller, tools/model/data are the model layer; testability of each layer is the payoff.

Task 18: Reasoning trace panel + edge cases + example polish

- Goal: A collapsible "trace" per answer (plan steps + tool calls + outputs from `execution_log`); explicit handling of empty results; curated example prompts.

- Concepts: Visible provenance — showing the tool log next to the answer is the user-facing proof of anti-hallucination; graders can see every number traces to a tool call. Empty-result handling — "no customers matched that filter" instead of a confusing blank/error. Quick prompts — a sidebar of starter questions ("Which customers are most likely to churn?" etc.) that double as your own smoke tests.

- Decisions: Trace panel hidden by default (collapsible) so the UX stays clean; empty results return a structured "no data" message rather than propagating an empty `DataFrame`.

- Architecture: The trace comes straight from `agent.run()`'s `execution_log` — no new plumbing needed, which is the payoff of capturing observations in Task 15.

- Theory: Explainable UI — transparency isn't just for the grader; it's the interface analog of the verify gate, turning "trust the model" into "inspect the evidence."

Phase G — Engineering Hygiene (Tasks 19–20)

Task 19: Tests + Docker

- Goal: tests/ with a few sanity tests per layer (data cleaning produces expected shape/encoding; `predict_churn_risk` returns valid schema for an ID and a dict; the exec tool runs a sample query and blocks import os; the validator catches an injected fake number). Then a working Dockerfile + docker-compose/run instructions.

- Concepts: Test pyramid — a handful of targeted tests per layer beats 50 meaningless ones; the exec-tool security test (denies import os) is the most valuable line in the suite. Reproducibility — the Dockerfile pip install `-r requirements.txt` + streamlit run `app.py`; test docker build && docker run locally — the guide explicitly warns not to assume it works.

- Decisions: Keep tests pure-pytest, no external calls (mock the LLM client); Docker image small (slim base, `--no-cache` pip install).

- Architecture: Tests import each layer independently — which only works because the layers are cleanly separated; Docker wraps the whole thing.

- Theory: Continuous verification — tests are the cheap cousin of the agent's self-check: both are "prove it before you ship it."

Task 20: README, reflection, time log, deploy, final audit

- Goal: README (what you built, data issues found + how handled, metric justification, how planning/verification work, AI-tool-use disclosure — the brief requires disclosing you used Claude/Copilot), half-page reflection, honest time log, GitHub public repo, Streamlit Community Cloud / ngrok deployment, then run the full "Done" checklist.

- Concepts: Evaluative writing — the README is doing real grading work; the reflection that admits what was hard and what you'd redo reads more credible than a list of wins. The checklist as a contract — every checkbox is a requirement, not a stretch.

- Decisions: Deploy to Streamlit Community Cloud (~10 min, free) as primary; ngrok only as backup. Rehearse the six follow-up questions out loud, from memory of your own code — especially "point to the verification line."

- Architecture: Everything up to now was preparation for this: the commit history is your process proof, the README your argument, the trace panel your evidence.

- Theory: Communication as engineering — the brief says it evaluates "how you think, not just what you ship"; your written artifacts are where that thinking is demonstrated.